

Activity_Course 2 Automatidata project lab

September 28, 2025

1 Automatidata project

Course 2 - Get Started with Python

Welcome to the Automatidata Project!

You have just started as a data professional in a fictional data consulting firm, Automatidata. Their client, the New York City Taxi and Limousine Commission (New York City TLC), has hired the Automatidata team for its reputation in helping their clients develop data-based solutions.

The team is still in the early stages of the project. Previously, you were asked to complete a project proposal by your supervisor, DeShawn Washington. You have received notice that your project proposal has been approved and that New York City TLC has given the Automatidata team access to their data. To get clear insights, New York City TLC's data must be analyzed, key variables identified, and the dataset ensured it is ready for analysis.

A notebook was structured and prepared to help you in this project. Please complete the following questions.

2 Course 2 End-of-course project: Inspect and analyze data

In this activity, you will examine data provided and prepare it for analysis. This activity will help ensure the information is,

1. Ready to answer questions and yield insights
2. Ready for visualizations
3. Ready for future hypothesis testing and statistical methods

The purpose of this project is to investigate and understand the data provided.

The goal is to use a dataframe constructed within Python, perform a cursory inspection of the provided dataset, and inform team members of your findings.

This activity has three parts:

Part 1: Understand the situation * Prepare to understand and organize the provided taxi cab dataset and information.

Part 2: Understand the data

- Create a pandas dataframe for data learning, future exploratory data analysis (EDA), and statistical activities.
- Compile summary information about the data to inform next steps.

Part 3: Understand the variables

- Use insights from your examination of the summary data to guide deeper investigation into specific variables.

Follow the instructions and answer the following questions to complete the activity. Then, you will complete an Executive Summary using the questions listed on the PACE Strategy Document.

Be sure to complete this activity before moving on. The next course item will provide you with a completed exemplar to compare to your own work.

3 Identify data types and relevant variables using Python

4 PACE stages

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.

4.1 PACE: Plan

Consider the questions in your PACE Strategy Document and those below to craft your response:

4.1.1 Task 1. Understand the situation

- How can you best prepare to understand and organize the provided taxi cab information?

To prepare for understanding and organizing the TLC data, I'll start by reviewing the project's goal — predicting taxi fares using real-world trip data. Focus on what variables are available and relevant, then outline the steps I'll take: loading the data, inspecting its structure, cleaning it, and preparing it for analysis.

- What follow-along and self-review codebooks will help you perform this work?

The most useful resources will be the course's Jupyter notebook, any sample code provided, the **pandas** and **Numpy** reference guides, and the data dictionary. These will guide me through loading, inspecting, and working with the data efficiently.

- What are some additional activities a resourceful learner would perform before starting to code?

Before coding, it's helpful to explore the dataset structure using `.head()`, `.info()` to spot any issues like missing values or incorrect types. We should also probably think about what features

are likely to impact fare amounts and consider reviewing similar examples of fare prediction to get ideas for how to approach this analysis.

4.2 PACE: Analyze

Consider the questions in your PACE Strategy Document to reflect on the Analyze stage.

4.2.1 Task 2a. Build dataframe

Create a pandas dataframe for data learning, and future exploratory data analysis (EDA) and statistical activities.

Code the following,

- import pandas as **pd**. pandas is used for buidling dataframes.
- import numpy as **np**. numpy is imported with pandas
- **df** = **pd.read_csv('Datasets\NYC taxi data.csv')**

Note: pair the data object name **df** with pandas functions to manipulate data, such as **df.groupby()**.

Note: As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```
[1]: #Import libraries and packages listed above
import pandas as pd
import numpy as np

# Load dataset into dataframe
df = pd.read_csv('2017_Yellow_Taxi_Trip_Data.csv')
print("done")
```

done

4.2.2 Task 2b. Understand the data - Inspect the data

View and inspect summary information about the dataframe by coding the following:

1. **df.head(10)**
2. **df.info()**
3. **df.describe()**

Consider the following two questions:

Question 1: When reviewing the **df.info()** output, what do you notice about the different variables? Are there any null values? Are all of the variables numeric? Does anything else stand out?

Question 2: When reviewing the `df.describe()` output, what do you notice about the distributions of each variable? Are there any questionable values?

Answer 1: From the `df.info()`, We noticed that some variable(column) names are inconsistent, identifier has no name. There are no missing or null values which is good. Not variables are numeric. Initially it's the variable names and datatypes, specially for dates which will be probably useful for calculating ride durations and trends over time.

Answer 2: From the `df.describe()`, We noticed that there are negative values for `fare_amount` and `total_amount`. Also, `passenger_count` and `trip_distance` include zero values which is a bit suspicious.

```
[2]: df.head(10)
```

```
[2]:   Unnamed: 0  VendorID  tpep_pickup_datetime  tpep_dropoff_datetime  \
0      24870114         2  03/25/2017 8:55:43 AM  03/25/2017 9:09:47 AM  \
1      35634249         1  04/11/2017 2:53:28 PM  04/11/2017 3:19:58 PM  \
2      106203690         1  12/15/2017 7:26:56 AM  12/15/2017 7:34:08 AM  \
3      38942136         2  05/07/2017 1:17:59 PM  05/07/2017 1:48:14 PM  \
4      30841670         2  04/15/2017 11:32:20 PM  04/15/2017 11:49:03 PM  \
5      23345809         2  03/25/2017 8:34:11 PM  03/25/2017 8:42:11 PM  \
6      37660487         2  05/03/2017 7:04:09 PM  05/03/2017 8:03:47 PM  \
7      69059411         2  08/15/2017 5:41:06 PM  08/15/2017 6:03:05 PM  \
8       8433159         2  02/04/2017 4:17:07 PM  02/04/2017 4:29:14 PM  \
9      95294817         1  11/10/2017 3:20:29 PM  11/10/2017 3:40:55 PM  \

      passenger_count  trip_distance  RatecodeID  store_and_fwd_flag  \
0                   6           3.34           1                   N  \
1                   1           1.80           1                   N  \
2                   1           1.00           1                   N  \
3                   1           3.70           1                   N  \
4                   1           4.37           1                   N  \
5                   6           2.30           1                   N  \
6                   1          12.83           1                   N  \
7                   1           2.98           1                   N  \
8                   1           1.20           1                   N  \
9                   1           1.60           1                   N  \

      PULocationID  DOLocationID  payment_type  fare_amount  extra  mta_tax  \
0              100           231            1          13.0    0.0    0.5  \
1              186           43             1          16.0    0.0    0.5  \
2              262          236            1           6.5    0.0    0.5  \
3              188           97            1          20.5    0.0    0.5  \
4               4          112            2          16.5    0.5    0.5  \
5             161          236            1           9.0    0.5    0.5  \
6              79          241            1          47.5    1.0    0.5  \
7             237          114            1          16.0    1.0    0.5  \
8             234          249            2           9.0    0.0    0.5  \
```

```
9          239          237          1          13.0          0.0          0.5
```

```

tip_amount  tolls_amount  improvement_surcharge  total_amount
0          2.76          0.0                   0.3          16.56
1          4.00          0.0                   0.3          20.80
2          1.45          0.0                   0.3           8.75
3          6.39          0.0                   0.3          27.69
4          0.00          0.0                   0.3          17.80
5          2.06          0.0                   0.3          12.36
6          9.86          0.0                   0.3          59.16
7          1.78          0.0                   0.3          19.58
8          0.00          0.0                   0.3           9.80
9          2.75          0.0                   0.3          16.55

```

```
[3]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            22699 non-null  int64
1   VendorID                             22699 non-null  int64
2   tpep_pickup_datetime                 22699 non-null  object
3   tpep_dropoff_datetime                22699 non-null  object
4   passenger_count                       22699 non-null  int64
5   trip_distance                        22699 non-null  float64
6   RatecodeID                           22699 non-null  int64
7   store_and_fwd_flag                   22699 non-null  object
8   PULocationID                         22699 non-null  int64
9   DOLocationID                         22699 non-null  int64
10  payment_type                          22699 non-null  int64
11  fare_amount                           22699 non-null  float64
12  extra                                 22699 non-null  float64
13  mta_tax                              22699 non-null  float64
14  tip_amount                           22699 non-null  float64
15  tolls_amount                          22699 non-null  float64
16  improvement_surcharge                 22699 non-null  float64
17  total_amount                          22699 non-null  float64
dtypes: float64(8), int64(7), object(3)
memory usage: 3.1+ MB

```

```
[4]: df.describe()
```

```

[4]:          Unnamed: 0      VendorID  passenger_count  trip_distance  \
count  2.269900e+04  22699.000000    22699.000000    22699.000000
mean   5.675849e+07    1.556236      1.642319      2.913313

```

std	3.274493e+07	0.496838	1.285231	3.653171
min	1.212700e+04	1.000000	0.000000	0.000000
25%	2.852056e+07	1.000000	1.000000	0.990000
50%	5.673150e+07	2.000000	1.000000	1.610000
75%	8.537452e+07	2.000000	2.000000	3.060000
max	1.134863e+08	2.000000	6.000000	33.960000

	RatecodeID	PULocationID	DOLocationID	payment_type	fare_amount \
count	22699.000000	22699.000000	22699.000000	22699.000000	22699.000000
mean	1.043394	162.412353	161.527997	1.336887	13.026629
std	0.708391	66.633373	70.139691	0.496211	13.243791
min	1.000000	1.000000	1.000000	1.000000	-120.000000
25%	1.000000	114.000000	112.000000	1.000000	6.500000
50%	1.000000	162.000000	162.000000	1.000000	9.500000
75%	1.000000	233.000000	233.000000	2.000000	14.500000
max	99.000000	265.000000	265.000000	4.000000	999.990000

	extra	mta_tax	tip_amount	tolls_amount \
count	22699.000000	22699.000000	22699.000000	22699.000000
mean	0.333275	0.497445	1.835781	0.312542
std	0.463097	0.039465	2.800626	1.399212
min	-1.000000	-0.500000	0.000000	0.000000
25%	0.000000	0.500000	0.000000	0.000000
50%	0.000000	0.500000	1.350000	0.000000
75%	0.500000	0.500000	2.450000	0.000000
max	4.500000	0.500000	200.000000	19.100000

	improvement_surcharge	total_amount
count	22699.000000	22699.000000
mean	0.299551	16.310502
std	0.015673	16.097295
min	-0.300000	-120.300000
25%	0.300000	8.750000
50%	0.300000	11.800000
75%	0.300000	17.800000
max	0.300000	1200.290000

4.2.3 Initial Insights from df.info()

Summary of Preparation Tasks

1. Convert Datetime Columns

Convert `tpep_pickup_datetime` and `tpep_dropoff_datetime` from object to datetime format using `pd.to_datetime()`.

2. Convert to Categorical Data Types

Convert `VendorID`, `RatecodeID`, `store_and_fwd_flag`, and `payment_type` to the category

data type where appropriate.

3. Inspect for Anomalies (mentioned in *Initial Insights from df.describe()*)

Check for invalid or suspicious values like:

- Zero or negative trip_distance, fare_amount, or passenger_count
- Trips with the same pickup and dropoff time

4. Drop or Rename Unnamed: 0

This appears to be an old index column from a CSV import. Rename it to record_id or drop it if not needed.

5. Standardize Column Names

Rename all columns to lowercase with underscores for consistency and better readability.

Original Column	Suggested Name
Unnamed: 0	record_id
VendorID	vendor_id
tpep_pickup_datetime	pickup_datetime
tpep_dropoff_datetime	dropoff_datetime
RatecodeID	rate_code_id
store_and_fwd_flag	store_and_forward_flag
PULocationID	pickup_location_id
DOLocationID	dropoff_location_id

Suggested Column Name Replacements

This structure will improve readability, consistency, and maintainability of your analysis.

```
[5]: # Renaming columns for clarity and consistency
taxi_trip_df = df.rename(columns={'Unnamed: 0': 'record_id',
                                'VendorID': 'vendor_id',
                                'tpep_pickup_datetime': 'pickup_datetime',
                                'tpep_dropoff_datetime': 'dropoff_datetime',
                                'RatecodeID': 'rate_code_id',
                                'store_and_fwd_flag': 'store_and_forward_flag',
                                'PULocationID': 'pickup_location_id',
                                'DOLocationID': 'dropoff_location_id'})

# Changing datatype of obvious datetime fields
taxi_trip_df[['pickup_datetime', 'dropoff_datetime']] = taxi_trip_df[['pickup_datetime', 'dropoff_datetime']].apply(pd.to_datetime)
```

```
[6]: print(taxi_trip_df[['vendor_id']].value_counts())
      print(taxi_trip_df[['rate_code_id']].value_counts())
      print(taxi_trip_df[['payment_type']].value_counts())
      print(taxi_trip_df[['store_and_forward_flag']].value_counts())
```

```
vendor_id
2          12626
1          10073
dtype: int64
rate_code_id
1          22070
2           513
5           68
3           39
4            8
99            1
dtype: int64
payment_type
1          15265
2           7267
3           121
4            46
dtype: int64
store_and_forward_flag
N          22600
Y            99
dtype: int64
```

```
[7]: # Update observations having a `rate_code_id` of 99 to 6 (Voided trip), before
      ↪ converting to datatype category
      taxi_trip_df.loc[taxi_trip_df['rate_code_id'] == 99, 'rate_code_id'] = 6
      taxi_trip_df['rate_code_id'].value_counts()
```

```
[7]: 1    22070
      2     513
      5     68
      3     39
      4      8
      6      1
      Name: rate_code_id, dtype: int64
```

```
[8]: '''
      Converting `vendor_id`, `rate_code_id`, `payment_type`, and
      ↪ `store_and_forward_flag` to category:
      * Saves memory
      * Speeds up operations like grouping, filtering
      * Makes the meaning of the data clearer (they're not meant for math)
```

```
'''
taxi_trip_df['vendor_id'] = taxi_trip_df['vendor_id'].astype('category')
taxi_trip_df['rate_code_id'] = taxi_trip_df['rate_code_id'].astype('category')
taxi_trip_df['payment_type'] = taxi_trip_df['payment_type'].astype('category')
taxi_trip_df['store_and_forward_flag'] = taxi_trip_df['store_and_forward_flag'].
↳astype('category')
taxi_trip_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   record_id                            22699 non-null  int64
1   vendor_id                            22699 non-null  category
2   pickup_datetime                      22699 non-null  datetime64[ns]
3   dropoff_datetime                    22699 non-null  datetime64[ns]
4   passenger_count                      22699 non-null  int64
5   trip_distance                       22699 non-null  float64
6   rate_code_id                        22699 non-null  category
7   store_and_forward_flag              22699 non-null  category
8   pickup_location_id                  22699 non-null  int64
9   dropoff_location_id                 22699 non-null  int64
10  payment_type                        22699 non-null  category
11  fare_amount                         22699 non-null  float64
12  extra                              22699 non-null  float64
13  mta_tax                            22699 non-null  float64
14  tip_amount                         22699 non-null  float64
15  tolls_amount                       22699 non-null  float64
16  improvement_surcharge               22699 non-null  float64
17  total_amount                       22699 non-null  float64
dtypes: category(4), datetime64[ns](2), float64(8), int64(4)
memory usage: 2.5 MB
```

4.2.4 Initial Insights from df.describe()

General Overview The dataset contains 22,699 entries across all numeric columns, indicating no missing values.

Potential Data Issues

- **Invalid Values:** Some entries have `passenger_count` and `trip_distance` equal to 0, which are likely incorrect.
- **Negative Values:** `fare_amount`, `total_amount`, and related financial fields contain negative values, which may indicate errors or test data.
- **Unusual IDs:** `RatecodeID` has a maximum value of 99, which exceeds the typical range (usually 1–6), suggesting outliers or miscoded data.

- **Extreme Outliers:**
 - fare_amount maxes at **\$999.99**
 - trip_distance maxes at **34 miles**
 - tip_amount reaches **\$200**
 - total_amount hits **\$1200.29**

Typical Ride Stats

- **Median Fare:** \$9.50
- **Median Total Amount:** \$11.80
- **Trip Distance (Median):** 1.61 miles
- **Passenger Count (Median):** 1

Suggested Next Steps

- Filter out rows with zero or negative values for key fields like `passenger_count`, `trip_distance`, and `fare_amount`.
- Investigate and possibly exclude rows with extremely high fare or tip values.
- Validate the consistency and expected range of categorical IDs like `RatecodeID` and `payment_type` before analysis or modeling.

4.2.5 Task 2c. Understand the data - Investigate the variables

Sort and interpret the data table for two variables: `trip_distance` and `total_amount`.

Answer the following three questions:

Question 1: Sort your first variable (`trip_distance`) from maximum to minimum value, do the values seem normal?

Question 2: Sort by your second variable (`total_amount`), are any values unusual?

Question 3: Are the resulting rows similar for both sorts? Why or why not?

Answer 1: The values doesn't seem normal to me, the first 2 rows from the query has almost similar distances which was covered in just 40 mins. to 1 hour which is highly unlikely in a (taxi) city driving scenario in NYC.

Answer 2: The values are not normal as well, we have the highest which is \$1200, covering 2 miles distance in less than a minute - that's 160 mph. There's also a lot of 0 mile distances that have `total_amount` values.

Answer 3: They're not similar, since we're using different criteria for the 2 sorts. Also, we can't conclude if there's a correlation between distance and fare amount because some data are unusual or invalid.

```
[9]: # Adding a computed column - trip_duration (in minutes) which is derived from
      ↳ pickup_datetime and dropoff_datetime
taxi_trip_df['trip_duration'] = (
    (taxi_trip_df['dropoff_datetime'] - taxi_trip_df['pickup_datetime'])
    .dt.total_seconds() / 60
).round(2)

# And speed_mph = trip_distance / (trip_duration (in minutes) / 60)
taxi_trip_df['speed_mph'] = (
    taxi_trip_df['trip_distance'] / (taxi_trip_df['trip_duration'] / 60)
).round(2)
```

```
[10]: taxi_trip_df.head(10)
```

```
[10]:
```

	record_id	vendor_id	pickup_datetime	dropoff_datetime	\
0	24870114	2	2017-03-25 08:55:43	2017-03-25 09:09:47	
1	35634249	1	2017-04-11 14:53:28	2017-04-11 15:19:58	
2	106203690	1	2017-12-15 07:26:56	2017-12-15 07:34:08	
3	38942136	2	2017-05-07 13:17:59	2017-05-07 13:48:14	
4	30841670	2	2017-04-15 23:32:20	2017-04-15 23:49:03	
5	23345809	2	2017-03-25 20:34:11	2017-03-25 20:42:11	
6	37660487	2	2017-05-03 19:04:09	2017-05-03 20:03:47	
7	69059411	2	2017-08-15 17:41:06	2017-08-15 18:03:05	
8	8433159	2	2017-02-04 16:17:07	2017-02-04 16:29:14	
9	95294817	1	2017-11-10 15:20:29	2017-11-10 15:40:55	

	passenger_count	trip_distance	rate_code_id	store_and_forward_flag	\
0	6	3.34	1	N	
1	1	1.80	1	N	
2	1	1.00	1	N	
3	1	3.70	1	N	
4	1	4.37	1	N	
5	6	2.30	1	N	
6	1	12.83	1	N	
7	1	2.98	1	N	
8	1	1.20	1	N	
9	1	1.60	1	N	

	pickup_location_id	dropoff_location_id	payment_type	fare_amount	extra	\
0	100	231	1	13.0	0.0	
1	186	43	1	16.0	0.0	
2	262	236	1	6.5	0.0	
3	188	97	1	20.5	0.0	

4	4	112	2	16.5	0.5
5	161	236	1	9.0	0.5
6	79	241	1	47.5	1.0
7	237	114	1	16.0	1.0
8	234	249	2	9.0	0.0
9	239	237	1	13.0	0.0

	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	\
0	0.5	2.76	0.0	0.3	16.56	
1	0.5	4.00	0.0	0.3	20.80	
2	0.5	1.45	0.0	0.3	8.75	
3	0.5	6.39	0.0	0.3	27.69	
4	0.5	0.00	0.0	0.3	17.80	
5	0.5	2.06	0.0	0.3	12.36	
6	0.5	9.86	0.0	0.3	59.16	
7	0.5	1.78	0.0	0.3	19.58	
8	0.5	0.00	0.0	0.3	9.80	
9	0.5	2.75	0.0	0.3	16.55	

	trip_duration	speed_mph
0	14.07	14.24
1	26.50	4.08
2	7.20	8.33
3	30.25	7.34
4	16.72	15.68
5	8.00	17.25
6	59.63	12.91
7	21.98	8.13
8	12.12	5.94
9	20.43	4.70

```
[11]: # Sort the data by trip distance from maximum to minimum value
trip_distance_sorted_df = taxi_trip_df.sort_values('trip_distance',
↪ascending=False)
trip_distance_sorted_df.head(20)[['record_id',
                                'trip_distance',
                                'trip_duration',
                                'speed_mph',
                                'pickup_location_id',
                                'dropoff_location_id',
                                'total_amount',
                                'payment_type']]
```

```
[11]:      record_id  trip_distance  trip_duration  speed_mph  pickup_location_id \
9280    51810714         33.96         39.22        51.95             132
13861    40523668         33.92         60.15        33.84             229
6064     49894023         32.72         67.48        29.09             138
```

10291	76319330	31.95	37.90	50.58	138
29	94052446	30.83	209.17	8.84	132
18130	90375786	30.50	87.80	20.84	132
5792	68023798	30.33	63.50	28.66	132
15350	77309977	28.23	49.75	34.05	13
10302	43431843	28.20	51.70	32.73	90
2592	51094874	27.97	50.37	33.32	261
20612	67238347	27.88	39.07	42.82	132
1908	81396251	27.34	36.62	44.80	132
20545	71466504	27.20	45.27	36.05	132
4138	100097208	26.86	72.78	22.14	132
15169	71224174	26.54	48.67	32.72	132
1496	27465882	26.39	66.67	23.75	132
7217	13572254	26.20	61.05	25.75	13
908	25075013	26.12	37.10	42.24	138
19483	34486843	26.12	159.32	9.84	132
4715	78418799	25.86	58.00	26.75	90

	dropoff_location_id	total_amount	payment_type
9280	265	150.30	2
13861	265	258.21	1
6064	1	179.06	1
10291	265	131.80	2
29	23	111.38	1
18130	220	119.31	1
5792	158	73.20	1
15350	132	62.96	1
10302	132	70.27	1
2592	132	63.06	2
20612	181	89.16	1
1908	181	88.56	1
20545	54	88.55	1
4138	52	94.75	1
15169	195	86.76	1
1496	261	70.27	1
7217	132	74.80	1
908	265	121.56	1
19483	48	58.56	2
4715	265	85.06	2

```
[12]: # Sort the data by total amount and print the top 20 values
trip_amount_sorted_df = taxi_trip_df.sort_values('total_amount',
↪ascending=False)
trip_amount_sorted_df.head(20)[['record_id',
                                'fare_amount',
                                'total_amount',
                                'trip_distance',
```

```
'trip_duration',
'speed_mph',
'pickup_location_id',
'dropoff_location_id',
'payment_type']]
```

```
[12]:      record_id  fare_amount  total_amount  trip_distance  trip_duration  \
8476      11157412         999.99        1200.29           2.60           0.97
20312     107558404         450.00         450.30           0.00           0.15
13861      40523668         200.01         258.21          33.92          60.15
12511     107108848         175.00         233.74           0.00           0.30
15474      55538852         200.00         211.80           0.00           0.08
6064       49894023         107.00         179.06          32.72          67.48
16379     101198443         140.00         157.06          25.50          50.57
3582      111653084         152.00         152.30           7.30           0.68
11269      51920669         120.00         151.82           0.00           0.92
9280       51810714         150.00         150.30          33.96          39.22
1928       51087145         120.00         137.80          12.50          48.70
10291      76319330         131.00         131.80          31.95          37.90
6708       91660295         100.00         126.00           0.32           0.05
11608     107690629          99.50         123.30          23.00         101.00
908        25075013         100.00         121.56          26.12          37.10
7281      111091850         100.00         120.96           0.00           0.15
18130      90375786          90.50         119.31          30.50          87.80
13621      93330154         105.00         115.94          19.80          46.60
13359       3055315          75.00         111.95           0.00           0.33
29         94052446          80.00         111.38          30.83         209.17
```

```
      speed_mph  pickup_location_id  dropoff_location_id  payment_type
8476        160.82                226                226             1
20312          0.00                265                265             2
13861         33.84                229                265             1
12511          0.00                265                265             1
15474          0.00                265                265             1
6064         29.09                138                 1             1
16379         30.26                132                265             2
3582        644.12                 1                 1             1
11269          0.00                265                265             1
9280         51.95                132                265             2
1928         15.40                211                265             1
10291         50.58                138                265             2
6708        384.00                264                 83             1
11608         13.66                151                 1             1
908         42.24                138                265             1
7281          0.00                265                265             1
18130         20.84                132                220             1
13621         25.49                265                230             1
```

13359	0.00	1	1	1
29	8.84	132	23	1

[13]: # Sort the data by total amount and print the bottom 20 values

```
trip_amount_sorted_df.tail(20)[['record_id',
                                'fare_amount',
                                'total_amount',
                                'trip_distance',
                                'trip_duration',
                                'speed_mph',
                                'pickup_location_id',
                                'dropoff_location_id',
                                'payment_type']]
```

```
[13]:
```

	record_id	fare_amount	total_amount	trip_distance	trip_duration	\
14283	37675840	0.01	0.31	0.00	0.17	
19067	58713019	0.00	0.30	0.10	0.83	
10506	26005024	0.00	0.00	0.00	0.03	
5722	49670364	0.00	0.00	0.00	0.03	
4402	108016954	0.00	0.00	7.06	40.95	
22566	19022898	0.00	0.00	0.00	0.05	
1646	57337183	-2.50	-3.30	0.04	0.62	
18565	43859760	-3.00	-3.80	0.10	1.03	
314	105454287	-2.50	-3.80	0.12	0.48	
5758	833948	-2.50	-3.80	0.02	0.27	
5448	28459983	-3.50	-4.30	0.25	2.22	
4423	97329905	-3.00	-4.30	0.06	1.33	
10281	55302347	-2.50	-4.30	0.00	2.07	
8204	91187947	-3.50	-4.80	0.41	2.38	
20317	75926915	-3.50	-4.80	0.24	2.25	
11204	58395501	-4.50	-5.30	0.64	2.85	
14714	109276092	-4.00	-5.30	0.40	3.17	
17602	24690146	-4.00	-5.80	0.46	3.60	
20698	14668209	-4.50	-5.80	0.70	3.80	
12944	29059760	-120.00	-120.30	0.17	1395.68	

	speed_mph	pickup_location_id	dropoff_location_id	payment_type
14283	0.00	146	146	3
19067	7.23	261	13	3
10506	0.00	264	193	1
5722	0.00	264	193	1
4402	10.34	263	169	2
22566	0.00	264	193	1
1646	3.87	79	79	3
18565	5.83	230	163	3
314	15.00	161	161	3
5758	4.44	170	170	3

5448	6.76	90	68	3
4423	2.71	237	237	4
10281	0.00	238	238	4
8204	10.34	236	237	3
20317	6.40	116	116	4
11204	13.47	50	48	3
14714	7.57	164	161	4
17602	7.67	87	45	4
20698	11.05	65	25	4
12944	0.01	138	138	4

```
[14]: # How many of each payment type are represented in the data?
taxi_trip_df[['payment_type']].value_counts()
```

```
[14]: payment_type
1          15265
2           7267
3           121
4             46
dtype: int64
```

According to the data dictionary, the payment method was encoded as follows:

1 = Credit card
2 = Cash
3 = No charge
4 = Dispute
5 = Unknown
6 = Voided trip

```
[15]: # What is the average tip for trips paid for with credit card?
print(f"Average Credit Card Tip: {taxi_trip_df[taxi_trip_df['payment_type'] == 1]['tip_amount'].mean()}")

# What is the average tip for trips paid for with cash?
print(f"Average Cash Tip: {taxi_trip_df[taxi_trip_df['payment_type'] == 2]['tip_amount'].mean()}")
```

Average Credit Card Tip: 2.7298001965279934
Average Cash Tip: 0.0

```
[16]: # How many times is each vendor ID represented in the data?
vendor_counts = taxi_trip_df[['vendor_id']].value_counts()
vendor_counts = vendor_counts.sort_index(ascending=True)
vendor_counts
```

```
[16]: vendor_id
1          10073
```

```
2          12626
dtype: int64
```

```
[17]: # What is the mean total amount for each vendor?
vendor_group = taxi_trip_df.groupby('vendor_id')
print(vendor_group[['total_amount']].sum(numeric_only=True))
print(vendor_group[['total_amount']].mean(numeric_only=True))
```

```
          total_amount
vendor_id
1          164170.95
2          206061.14
          total_amount
vendor_id
1          16.298119
2          16.320382
```

```
[18]: # Filter the data for credit card payments only
trip_cc_only_df = taxi_trip_df[taxi_trip_df['payment_type'] == 1]

# Filter the credit-card-only data for passenger count only
trip_cc_only_df[['passenger_count']].value_counts().sort_index(ascending=True)
```

```
[18]: passenger_count
0          27
1        10977
2         2168
3          600
4          267
5          775
6          451
dtype: int64
```

```
[19]: # Calculate the average tip amount for each passenger count (credit card
      ↪ payments only)
trip_cc_tip_summary = trip_cc_only_df.groupby('passenger_count')['tip_amount'].
      ↪agg(
    count='count',
    total_tip='sum',
    average_tip='mean'
)
trip_cc_tip_summary
```

```
[19]:          count  total_tip  average_tip
passenger_count
0           27       70.48     2.610370
1        10977    29799.05     2.714681
```

2	2168	6135.33	2.829949
3	600	1636.08	2.726800
4	267	696.27	2.607753
5	775	2141.05	2.762645
6	451	1192.14	2.643326

Question 1: Will the available information be sufficient to achieve the goal based on your intuition and the analysis of the variables?

Question 2: How would you build summary dataframe statistics and assess the min and max range of the data?

Question 3: Do the averages of any of the data variables look unusual? Can you describe the interval data?

Answer 1: Yes, the available data includes all the key details needed to estimate fares ahead of time. With pickup and dropoff times, trip distance, and fare amount, we have enough context to identify patterns and make informed predictions for future trips. Some cleaning may be needed to improve accuracy, but the core data is in place.

Answer 2: You can use the `.describe()` method in pandas to generate a summary of each numeric column — including count, mean, min, max, and quartiles. You could also use `.min()` and `.max()` for specific variables you want to inspect, or spot outliers or incorrect entries.

Answer 3: Some average values may look unusual — for example, if the average fare or distance is unexpectedly low or high, or even negative, it could point to invalid or outlier trips. There isn't really any interval data in this dataset. Most of the numbers — like trip distance, fare amount, and trip duration (which we derived from pick-up time and drop-off time) — all have a clear starting point of zero and can be compared directly. For example, a 10-mile trip is twice as long as a 5-mile trip. That kind of data is more specific than interval data and is often called ratio data, which is better for doing calculations like averages or comparisons.

4.3 PACE: Construct

Note: The Construct stage does not apply to this workflow. The PACE framework can be adapted to fit the specific requirements of any project.

4.4 PACE: Execute

Consider the questions in your PACE Strategy Document and those below to craft your response.

Question 1: Given your current knowledge of the data, what would you initially recommend to your manager to investigate further prior to performing exploratory data analysis?

Question 2: What data initially presents as containing anomalies?

Question 3: What additional types of data could strengthen this dataset?

Answer 1: We would recommend checking for trips that have suspicious values — like zero or negative fare amounts, zero passengers, or trips with very short or very long durations. These could be errors, and they may affect any future analysis. Also, we'd suggest reviewing whether pickup

and dropoff locations are coded properly and if we need to map them to actual neighborhoods for clearer insights.

Answer 2: Some examples of data that look unusual: * Fare Amounts: Some trips may have a negative fare, 0 fare or extremely high fares (1200) that seem unrealistic. * Trip Distance: Values of 0 miles with a valid fare amount could be mistakes. * Trip Duration: Trips lasting just a few seconds or several hours (23) may not be valid, especially in a typical city (NYC) environment. * Passenger Count: Some trips show 0 passengers, which doesn't make sense for a taxi ride.

These kinds of values may be outliers or data entry errors and may need to be filtered or handled before doing deeper analysis.

Answer 3: A few extra types of information could improve the dataset: * Traffic conditions at the time of the trip — this could explain longer durations or higher fares. * Surge pricing or fare rules — if available, these would help explain fare variations. * Weather data — this can a bit excess maybe, but rainy or snowy days might affect trip time and demand.

Adding this kind of context would make fare estimation more accurate and insightful.

4.4.1 Given your efforts, what can you summarize for DeShawn and the data team?

Note for Learners: Your notebook should contain data that can address Luana's requests. Which two variables are most helpful for building a predictive model for the client: NYC TLC?

The taxi trip data is now organized and ready for deeper analysis. I've cleaned column names, converted data types, and added some derived variables (`trip_duration` and `speed_mph`). Some entries with negative or zero fares, extreme durations, or missing passengers may need review and further cleaning. The most important variable is `fare_amount`, as this is the value we aim to estimate. Variables like `trip_distance` or `trip_duration` follows as either of these will be key in helping predict fare amounts based on past trip patterns.

```
[20]: taxi_trip_df.head(10)
```

```
[20]:
```

	record_id	vendor_id	pickup_datetime	dropoff_datetime	\
0	24870114	2	2017-03-25 08:55:43	2017-03-25 09:09:47	
1	35634249	1	2017-04-11 14:53:28	2017-04-11 15:19:58	
2	106203690	1	2017-12-15 07:26:56	2017-12-15 07:34:08	
3	38942136	2	2017-05-07 13:17:59	2017-05-07 13:48:14	
4	30841670	2	2017-04-15 23:32:20	2017-04-15 23:49:03	
5	23345809	2	2017-03-25 20:34:11	2017-03-25 20:42:11	
6	37660487	2	2017-05-03 19:04:09	2017-05-03 20:03:47	
7	69059411	2	2017-08-15 17:41:06	2017-08-15 18:03:05	
8	8433159	2	2017-02-04 16:17:07	2017-02-04 16:29:14	
9	95294817	1	2017-11-10 15:20:29	2017-11-10 15:40:55	

	passenger_count	trip_distance	rate_code_id	store_and_forward_flag	\
0	6	3.34	1		N
1	1	1.80	1		N

2	1	1.00	1	N
3	1	3.70	1	N
4	1	4.37	1	N
5	6	2.30	1	N
6	1	12.83	1	N
7	1	2.98	1	N
8	1	1.20	1	N
9	1	1.60	1	N

	pickup_location_id	dropoff_location_id	payment_type	fare_amount	extra	\
0	100	231	1	13.0	0.0	
1	186	43	1	16.0	0.0	
2	262	236	1	6.5	0.0	
3	188	97	1	20.5	0.0	
4	4	112	2	16.5	0.5	
5	161	236	1	9.0	0.5	
6	79	241	1	47.5	1.0	
7	237	114	1	16.0	1.0	
8	234	249	2	9.0	0.0	
9	239	237	1	13.0	0.0	

	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	\
0	0.5	2.76	0.0	0.3	16.56	
1	0.5	4.00	0.0	0.3	20.80	
2	0.5	1.45	0.0	0.3	8.75	
3	0.5	6.39	0.0	0.3	27.69	
4	0.5	0.00	0.0	0.3	17.80	
5	0.5	2.06	0.0	0.3	12.36	
6	0.5	9.86	0.0	0.3	59.16	
7	0.5	1.78	0.0	0.3	19.58	
8	0.5	0.00	0.0	0.3	9.80	
9	0.5	2.75	0.0	0.3	16.55	

	trip_duration	speed_mph
0	14.07	14.24
1	26.50	4.08
2	7.20	8.33
3	30.25	7.34
4	16.72	15.68
5	8.00	17.25
6	59.63	12.91
7	21.98	8.13
8	12.12	5.94
9	20.43	4.70

```
[21]: taxi_trip_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   record_id                            22699 non-null  int64
1   vendor_id                            22699 non-null  category
2   pickup_datetime                      22699 non-null  datetime64[ns]
3   dropoff_datetime                    22699 non-null  datetime64[ns]
4   passenger_count                      22699 non-null  int64
5   trip_distance                       22699 non-null  float64
6   rate_code_id                        22699 non-null  category
7   store_and_forward_flag              22699 non-null  category
8   pickup_location_id                  22699 non-null  int64
9   dropoff_location_id                 22699 non-null  int64
10  payment_type                        22699 non-null  category
11  fare_amount                         22699 non-null  float64
12  extra                              22699 non-null  float64
13  mta_tax                            22699 non-null  float64
14  tip_amount                         22699 non-null  float64
15  tolls_amount                       22699 non-null  float64
16  improvement_surcharge               22699 non-null  float64
17  total_amount                       22699 non-null  float64
18  trip_duration                      22699 non-null  float64
19  speed_mph                          22673 non-null  float64
dtypes: category(4), datetime64[ns](2), float64(10), int64(4)
memory usage: 2.9 MB

```

```
[22]: taxi_trip_df.describe()
```

```

[22]:
      record_id  passenger_count  trip_distance  pickup_location_id \
count  2.269900e+04      22699.000000      22699.000000      22699.000000
mean    5.675849e+07          1.642319          2.913313          162.412353
std     3.274493e+07          1.285231          3.653171          66.633373
min     1.212700e+04          0.000000          0.000000           1.000000
25%     2.852056e+07          1.000000          0.990000          114.000000
50%     5.673150e+07          1.000000          1.610000          162.000000
75%     8.537452e+07          2.000000          3.060000          233.000000
max     1.134863e+08          6.000000          33.960000          265.000000

      dropoff_location_id  fare_amount      extra      mta_tax \
count      22699.000000      22699.000000      22699.000000      22699.000000
mean         161.527997         13.026629         0.333275         0.497445
std           70.139691         13.243791         0.463097         0.039465
min            1.000000        -120.000000        -1.000000        -0.500000
25%          112.000000          6.500000          0.000000          0.500000
50%          162.000000          9.500000          0.000000          0.500000

```

75%	233.000000	14.500000	0.500000	0.500000
max	265.000000	999.990000	4.500000	0.500000

	tip_amount	tolls_amount	improvement_surcharge	total_amount	\
count	22699.000000	22699.000000	22699.000000	22699.000000	
mean	1.835781	0.312542	0.299551	16.310502	
std	2.800626	1.399212	0.015673	16.097295	
min	0.000000	0.000000	-0.300000	-120.300000	
25%	0.000000	0.000000	0.300000	8.750000	
50%	1.350000	0.000000	0.300000	11.800000	
75%	2.450000	0.000000	0.300000	17.800000	
max	200.000000	19.100000	0.300000	1200.290000	

	trip_duration	speed_mph
count	22699.000000	22673.000000
mean	17.013766	13.821157
std	61.996476	153.386987
min	-16.980000	-20.140000
25%	6.650000	7.250000
50%	11.180000	9.840000
75%	18.380000	13.600000
max	1439.550000	16800.000000

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.